

NL2SQL

Project Teaching Guide

Natural Language → SQL using Transformers + Rules

CSE472 — Deep Learning for NLP | LPU

■ This guide explains the NL2SQL project from scratch — what it does, how each component works, why design decisions were made, and what to say if a faculty asks you about it. Read it once end-to-end and you'll be able to explain the whole project confidently.

1. The Big Picture

What does this project actually do?

Databases store information in tables — student records, marks, courses. To get data out, you normally need to write SQL. Most people can't write SQL. NL2SQL solves this: you type a plain English question,

and the system figures out the right SQL query and runs it for you.

■ You type: "show top 5 students by cgpa" ■ System returns: actual database rows — names, CGPAs, departments ■ SQL it ran behind the scenes: `SELECT * FROM students ORDER BY cgpa DESC LIMIT 5`

The Full Pipeline (in order):

Step	Component	What it does
1	FastAPI endpoint	Receives your English question via HTTP POST
2	NLPreprocessor	Cleans text, extracts keywords, detects intent (COUNT / TOP_N / etc.)
3	EmbeddingMatcher	Uses AI to find which table + columns your question refers to
4	SQLGenerator	Builds the SQL string from intent + table + columns
5	SQLExecutor	Validates and runs the SQL, returns results as JSON

2. The Preprocessor

`nlp/preprocessor.py` — Cleans text + detects intent

The preprocessor is the first thing that runs on your input. Its job is to take messy human text and extract structured information from it — specifically: what does the user want to do, and what numbers did they mention?

Step-by-step walkthrough:

Normalise	Convert to lowercase, strip punctuation, collapse spaces. "Show TOP 5 students!!" → "show top 5 students"
Tokenise	Split on spaces → ["show", "top", "5", "students"]
Extract keywords	Remove stopwords (the, a, show, get, list, display...) → ["top", "5", "students"]
Detect intent	Match against trigger word lists → TOP_N
Extract numbers	Find digits + word numbers → [5]

The 7 Intent Classes:

Intent	Trigger Words	SQL produced
COUNT	how many, count, total number	<code>SELECT COUNT(*) FROM ...</code>
AVERAGE	average, avg, mean	<code>SELECT AVG(col) FROM ...</code>

TOP_N	top, highest, best, maximum, max	ORDER BY col DESC LIMIT n
BOTTOM_N	bottom, lowest, worst, minimum, min	ORDER BY col ASC LIMIT n
FILTER_GT	above, greater than, more than, over	WHERE col > threshold
FILTER_LT	below, less than, under, low	WHERE col < threshold
SELECT	(default — nothing matched)	SELECT * FROM ... LIMIT 20

■ Why rule-based for intent? Because the 6-class label space is small and the trigger words are distinct enough. A machine learning classifier would give identical accuracy with extra complexity. Simple is better when it works.

3. The Embedding Matcher

[nlp/embeddings.py](#) — The AI brain of the system

This is the most technically interesting component. Its job is schema linking — figuring out which database table and columns the user's question is about, even when they use completely different words from the column names.

■ The problem: User says "show me students with good academic score" Column name in DB: cgpa ■ "academic score" and "cgpa" have zero words in common — but they mean the same thing. A keyword matcher would fail. The transformer handles this.

How it works — 3 stages:

Stage 1 — Synonym Expansion (before AI)

A 54-entry dictionary replaces common domain variants with their schema equivalent. This handles the obvious cases cheaply, so the AI only needs to handle the hard cases.

```
"grade" → "marks" | "branch" → "department"
"teacher" → "faculty" | "gpa" → "cgpa"
"percentage" → "attendance" | "professor" → "faculty"
"score" → "marks" | "kids" → "students"
```

Stage 2 — Transformer Encoding

The model used is **all-MiniLM-L6-v2** — a sentence transformer with 22 million parameters. It converts any text into a list of 384 numbers called an embedding vector. The key property: similar meaning = similar vectors.

```
"student academic score" → [0.21, -0.43, 0.87, 0.11, ...] (384 numbers)
"cumulative grade point average" → [0.19, -0.41, 0.85, 0.13, ...] (384 numbers)
↑ very similar!
```

■ Why 384 numbers? Each dimension captures some aspect of meaning — grammar, topic, sentiment, domain context. The model was trained on millions of sentence pairs to make semantically similar sentences have close vectors. We don't define what each dimension means — the model learns it.

Stage 3 — Cosine Similarity

Cosine similarity measures the angle between two vectors. If they point in the same direction (similar meaning), the score is close to 1.0 (100%). If unrelated, the score is near 0.

```
query = encode("show students with good academic score")

cosine(query, students_embedding) = 0.847 ← 84.7% ■ WINNER
cosine(query, marks_embedding) = 0.612 ← 61.2%
cosine(query, courses_embedding) = 0.431 ← 43.1%

→ Table selected: students
```

At Startup vs At Query Time

At Startup (once)	At Query Time (every request)
Load all-MiniLM-L6-v2 model (~1.2s)	Run synonym expansion (~0ms)
Encode all table descriptions	Encode user's query (~91ms)
Encode all column descriptions	Cosine similarity vs tables
Store in memory as numpy arrays	Cosine similarity vs columns
	Return top table + top 5 columns

4. The SQL Generator

`sql_generator/generator.py` — Builds SQL from intent + schema

The SQL Generator is pure deterministic logic — no AI at all. It takes three inputs (intent, matched table, extracted numbers) and fills in a template. Every possible output is valid SQL by construction — no hallucination possible.

Templates by Intent:

COUNT	SELECT COUNT(*) AS total FROM {table}
AVERAGE	SELECT AVG({col}) AS average_{col} FROM {table}

TOP_N	<code>SELECT * FROM {table} ORDER BY {col} DESC LIMIT {n}</code>
BOTTOM_N	<code>SELECT * FROM {table} ORDER BY {col} ASC LIMIT {n}</code>
FILTER_GT	<code>SELECT * FROM {table} WHERE {col} > {threshold}</code>
FILTER_LT	<code>SELECT * FROM {table} WHERE {col} < {threshold}</code>
SELECT	<code>SELECT * FROM {table} LIMIT 20 (or WHERE department='CSE' if dept detected)</code>

Full Example — end to end:

Input query: "top 3 students by attendance"

After preprocessing:

intent = TOP_N

numbers = [3]

keywords = ["students", "attendance"]

After embedding match:

table = "students" (84.7% confidence)

top col = "attendance" (matched by keyword)

SQL generated:

`SELECT * FROM students ORDER BY attendance DESC LIMIT 3`

Result: 3 rows — Anita C (95%), Priya K (91%), Karthik P (88%)

■ The explanation dict: alongside the SQL, the generator also returns a dictionary like {"intent": "TOP_N", "table": "students", "order_by": "attendance", "limit": 3}. This is returned in the API response so the frontend can show the user exactly why that query was constructed — full transparency.

5. Executor & Database

database/executor.py + schema.py

SQLExecutor — Safety first

Before any SQL touches the database, the executor runs a blocklist check. This prevents the system from ever modifying or deleting data, even if there's a bug somewhere in the generator.

```
BLOCKED = ["DROP", "DELETE", "INSERT", "UPDATE", "TRUNCATE", "ALTER", "CREATE", "EXEC"]
```

```
if any blocked keyword found → reject with error message
```

```
if SQL does not start with SELECT → reject
```

```
else → run via pandas.read_sql() → return rows as JSON
```

Database Schema (3 tables):

Table	Columns	Records
students	id, name, department, cgpa, attendance, semester	10 rows
marks	id, student_id (FK→students), subject, marks, semester	50 rows (5 subjects × 10 students)
courses	course_id, course_name, faculty, credits	5 rows

■■ Known limitation: The current system only handles single-table queries. A query like 'which students failed in Deep Learning' needs to JOIN students + marks — this is not supported yet and is the #1 failure mode (3 out of 7 wrong answers).

6. Results & Numbers

What to quote when asked about performance

We tested 60 hand-written queries — 10 per intent class. Three variations per class: direct phrasing, synonym variants, and semantically novel phrasings.

Intent	Queries	Execution Acc.	Schema Acc.	SQL Match	Avg Latency
COUNT	10	100%	100%	100%	142 ms
AVERAGE	10	90%	100%	90%	155 ms
TOP_N	10	90%	90%	90%	168 ms
BOTTOM_N	10	90%	100%	80%	171 ms
FILTER_GT	10	80%	90%	80%	163 ms
FILTER_LT	10	80%	80%	70%	177 ms
TOTAL	60	88.3%	93.3%	85.0%	163 ms

Key numbers to remember:

88.3%

Execution Accuracy — primary metric (53/60 queries correct)

93.3%

Schema Match Accuracy — correct table selected (56/60)

163 ms	Average end-to-end latency on CPU (no GPU)
+25 pts	Improvement from adding transformer over keyword-only baseline
84.7%	Average cosine similarity confidence for correct table matches
22M	Parameters in all-MiniLM-L6-v2 (tiny vs GPT-4's ~1.8 trillion)

7. Where It Fails & Why

7 wrong answers — what broke and why

7 out of 60 queries failed. Knowing exactly why is important — it shows you actually understand the system.

■ Failure 1 — Cross-Table JOIN (3 failures)

Query: "which students failed in Deep Learning" Needs: JOIN students + marks ON student_id, then filter WHERE subject='Deep Learning' AND marks < 50 What happened: System correctly picked the marks table but generated a single-table query. Returned mark records without student names. Root cause: No JOIN template exists in the generator. This is the biggest missing feature.

■ Failure 2 — Missing Synonyms (2 failures)

Query: "learners with poor scores" Problem: 'learners' was not in the 54-entry synonym dictionary. What happened: Embedding matcher still found the right table (students — semantically close), but column resolution defaulted to the wrong column. Fix: Just add 'learners → students' to the dictionary. No code change needed.

■ Failure 3 — Numeric Ambiguity (2 failures)

Query: "top 3.5 GPA students" Problem: Is 3.5 a count (top 3.5 students?) or a threshold (cgpa >= 3.5)? What happened: Number extractor pulled 3.5, TOP_N template used it as LIMIT (truncated to 3). User actually meant WHERE cgpa >= 3.5. Fix: Add logic to check if a float value plausibly serves as a filter threshold vs a result count.

8. Faculty Q&A;

What to say if you get asked these questions

Q: Why not just use ChatGPT or GPT-4 to generate the SQL?

A: Two reasons. First, cost and speed — GPT-4 API costs money per call and adds 1–3 seconds of latency. Our system runs on a regular CPU in 163ms. Second, transparency — if GPT-4 generates wrong SQL, you can't easily debug why. Our system is fully traceable: every decision maps to a specific component and you can see exactly why each SQL was produced.

Q: Why use a transformer at all if most of it is rule-based?

A: The transformer handles the one part where rules genuinely fail — schema linking with semantic variation. You cannot write a rule that maps 'academic score' to the cgpa column without a semantic model. The transformer gives us that generalisation cheaply (22M params, <90MB, no GPU) without needing to fine-tune a generative model.

Q: Why templates instead of a sequence-to-sequence model for SQL generation?

A: Templates guarantee syntactically valid SQL on every single call. A seq2seq model can hallucinate column names, forget closing parentheses, or generate semantically wrong queries in ways that are hard to catch. For a closed domain with six intent types, templates cover everything needed and are trivially auditable.

Q: What is cosine similarity and why use it?

A: Cosine similarity measures the angle between two vectors. If they point in the same direction, similarity is 1.0 (100%). If perpendicular, it's 0. We use it instead of dot product because it's magnitude-invariant — a long sentence and a short sentence with the same meaning get the same score, which is what we want when comparing variable-length descriptions.

Q: What's the biggest limitation of this system?

A: No support for multi-table JOIN queries. Three out of seven failures were caused by this. A query like 'which students failed in Deep Learning' needs to join the students and marks tables — the system can't do that yet. Adding a JOIN intent class with a two-table template is the highest priority next step.

Q: How is this different from traditional keyword-based NLIDBs?

A: Traditional systems like LADDER matched surface text patterns to query fragments — brittle and domain-locked. Our embedding matcher uses dense vector representations, so semantically similar text matches even without word overlap. We proved this with a baseline: the keyword-only version got 68.3% schema accuracy; adding the transformer got 93.3% — a 25-point improvement.

Q: What is all-MiniLM-L6-v2 and why this model specifically?

A: It's a distilled sentence transformer trained on natural language inference and paraphrase tasks. L6 means 6 transformer layers. It produces 384-dimensional embeddings. We chose it because: (1) 22M parameters — tiny compared to alternatives; (2) under 90MB model size; (3) ~91ms per sentence on CPU; (4) strong performance on semantic similarity benchmarks. Compared to all-mpnet-base-v2 which is better but 420MB and much slower — MiniLM-L6 was the right trade-off for a CPU deployment.

9. Quick Cheat Sheet

Everything you need to remember on one page

Component	What it does	Tech used
NLPPreprocessor	Normalise → tokenise → extract keywords → detect intent → extract numbers	Python regex + keyword rules
EmbeddingMatcher	Synonym expand → encode query → cosine similarity vs table/col embeddings → pick best match	all-MiniLM-L6-v2, scikit-learn cosine_similarity
SQLGenerator	Intent + table + numbers → fill SQL template → return SQL + explanation dict	Pure Python string formatting
SQLExecutor	Blocklist check → run SQL → return rows + columns + row_count as JSON	SQLAlchemy, pandas read_sql
FastAPI app	POST /query, GET /schema, GET /health	FastAPI, Pydantic, CORS middleware
Database	students (10), marks (50), courses (5) — SQLite file	SQLAlchemy ORM, SQLite

88.3% Execution Accuracy

93.3% Schema Match Accuracy

163 ms avg latency (CPU)

+25 pts over keyword baseline

22M params, <90MB model

60 test queries, 6 intent classes

3 × JOIN queries (no template)

2 × Missing synonyms (easy fix)

2 × Numeric ambiguity

Next step: add JOIN support